



DATA STRUCTURES

CONTENTS



7. Stack ADT

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. In this module we talk about a very important ADT – that is the Stack ADT.

Learning Objectives

The learning objectives of the introductory module are as follows:

- To understand the concept of a stack
- To appreciate the method of defining a Stack ADT
- To know about the different stack operations
- To understand some uses of stacks

7.1 Introduction

Before we go further let us look at the use of stacks in everyday life. In general a stack is a pile of objects, typically one that is neatly arranged. Figure 7.1 shows

stacks of notes, plates, sarees, chapathis, tyres and books that we see often see in our lives. In this module we will discuss the characteristics of the Stack used in the computing world.



Fig 7.1 Everyday Examples of Stacks

7.1.1 What is a stack?

A stack is a sequence of items that are accessible at only one end of the sequence. It is an ordered group of homogeneous items or elements. Elements are added to and removed from a specially designated end called the top of the stack (the most recently added items are at the top of the stack). The last element to be added is the first to be removed (LIFO: Last In, First Out). In other words, a stack is a **restricted linear list** in which all **additions and deletions are made at one end, the top**. If we insert a series of data items into a stack and then remove them, the order of the data is reversed. This **reversing attribute** is why stacks are known as **last in, first out (LIFO) data structures**. Figure 7.2 shows a computer stack which also indicates that an insertion at the Top into this restricted list is called Push and deletion also at the Top is called POP.

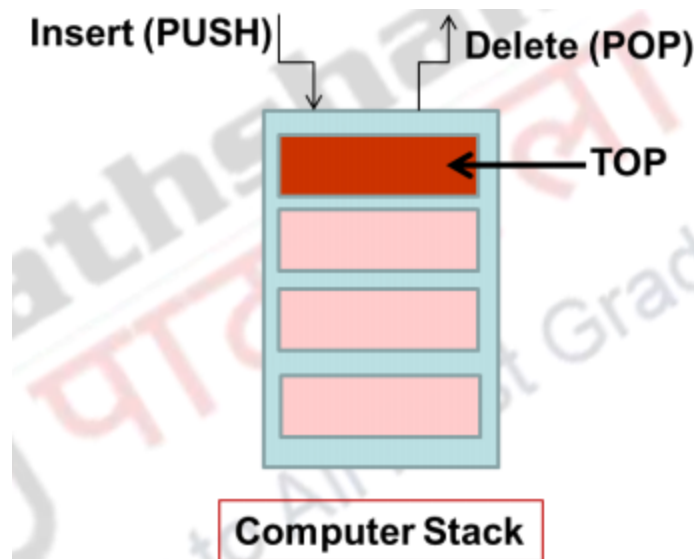


Fig 7.2 A Computer Stack

As already stated we assume that all elements of a stack are homogenous that is they are all of the same type. However the elements are unordered and are placed in the stack in the order in which they are inserted. Multiple Occurrences of the same elements is permitted as per the characteristics of the stack. The stack allows unidirectional access (LIFO(Last in First Out) / FILO(First In Last Out)) through the current Stack Top Pointer (Stack_Pointer).

7.1.2 Conceptual View of a Stack

The main operations associated with the stack are the Push (adding an element) and the Pop (removing an element) operations. Figure 7.3 shows the conceptual view of adding an element without going into the details. The blue plate (an object to be added needs to be provided) is to be added to the stack. The old Top pointed to the orange plate that was then at the top, now after the blue plate has been added the new top points to this plate. Please note that the bottom of the stack does not play any part in the addition process.

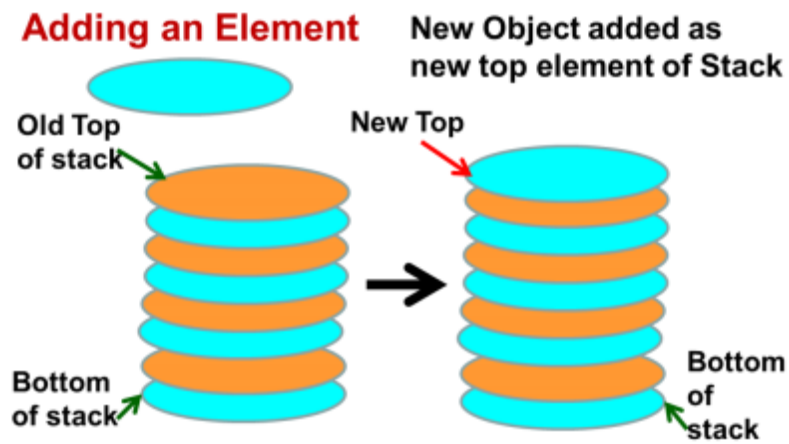


Figure 7.3 Adding an Element

Figure 7.4 shows the conceptual view of removing an element without going into the details. The blue plate that is on top of the stack is to be removed from the stack. In this case this is the only element that can be removed. While the old Top pointed to the blue plate that was then at the top, now after the blue plate has been removed the new top points to the orange plate that is now on top. Please note that the bottom of the stack does not play any part in the deletion process.

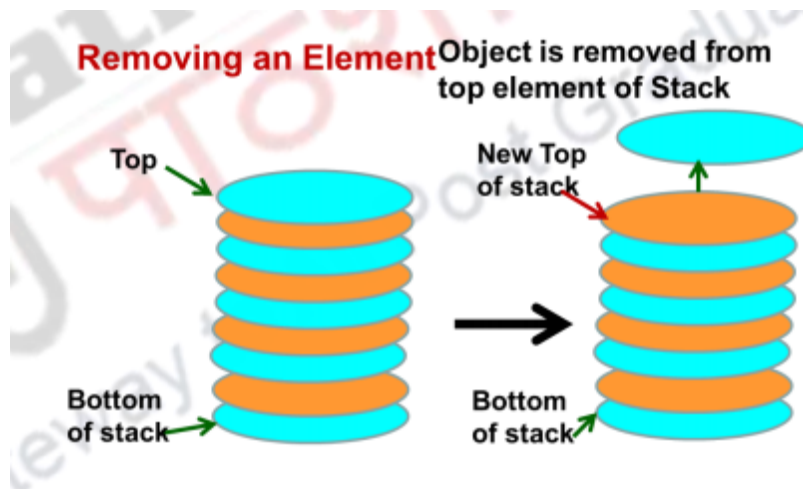


Figure 7.4 Removing an Element

7.2 Uses of Stack ADT

Stacks are used in many situations especially when there is a need to process elements in a reverse order to the order in which they arrive. This is an important requirement for many problems in the computing world such as using stacks when processing algebraic expressions, using stacks to search a flight map. Another important use of stacks is in tackling recursion. Later we will discuss the applications of stacks in detail.

7.3 Abstract Data Type – Stack

Let us now look at developing an ADT during the design of a solution to a problem. Consider entering text using a keyboard. Now if we make mistakes then use of backspace is required. For example if we first type **abcd**, and then by mistake type **d** again and then again, there is need to use two backspaces and then continue to type **efg**, and then **g** again by mistake, then again we need to use backspace. The sequence will be as seen in Figure 7.5. As you will realize, the letter typed last is the one to be removed by the backspace. We seek a programming solution to read these keystrokes. This is a direct mapping to a stack where also we delete elements entered last.

abcddd<-<-efgg<-

Figure 7.5 Keystrokes using a Keyboard

7.3.1 Stack ADT

The **Stack Abstract Data Type (Stack ADT)** is a collection of data together with the operations on that data. We will now formally define the **interface** of the collection. The interface of the collection tells us what we need to know in order to interact with it, i.e. how to use the operations associated with the stack. The stack can be specified by the user by defining

Max_Items – Maximum number of items that might be on the stack and

ItemType– Data type of the items on the stack.

7.3.2 ADT stack operations

The next step in defining the Stack ADT is to list the set of operations associated with the ADT. In this section we will list all the operations generally associated with

stacks. However we have to remember that the actual set of operations depend on the application for which the Stack ADT will be used and therefore the operations necessary for that application.

The first operation associated with any ADT is the creation of an empty ADT. Therefore we will first define the operation of creating an empty Stack. In some cases we may want to destroy the entire stack and hence that is the next operation we will define. A very important operation for many applications of stack is to have an operation to check whether a stack is empty. Then the stack will normally be associated with operations to add and remove elements from it. Finally there may be an operation to retrieve the top element from the stack without removing it. From the ADT perspective a program can use a stack independently of the stack's implementation.

Now let us describe in a little more in detail four important operations associated with the stack. They are:

- **bool isEmpty** – This is a Boolean operation that checks whether a stack is empty and signals a true value if the stack is empty.
- **push (ItemType newItem)** – This is the push operation that adds a new item newItem of type ItemType to the stack.
- **pop ()** – This is the pop operation where we remove from the Stack, the item that has been added most recently. In this definition of pop we do not output the item removed, we just change the contents of the stack.
- **top()** – This returns the last inserted element without removing it in other words gets the item that was added to stack most recently.

In addition to the most important operations described above, a set of auxiliary stack operations are also described below which will be part of the definition of a Stack ADT depending on the application. These auxiliary operations are described below:

- **size()** – This returns the number of elements that is currently available in the stack
- **Create** – This operation creates an Empty Stack with Stack Pointer (top) pointing to null and the number of Stack Elements set to 0

- **Check if Full** – This is a Boolean operation that will be true when the number of Elements = maximum number of elements allowed. However this operation is valid only for array based Implementation.

7.3.2.1 Creation of an Empty Stack

This **stack** operation `CreateStack(stackname)` creates an empty stack as shown in Figure 7.6.

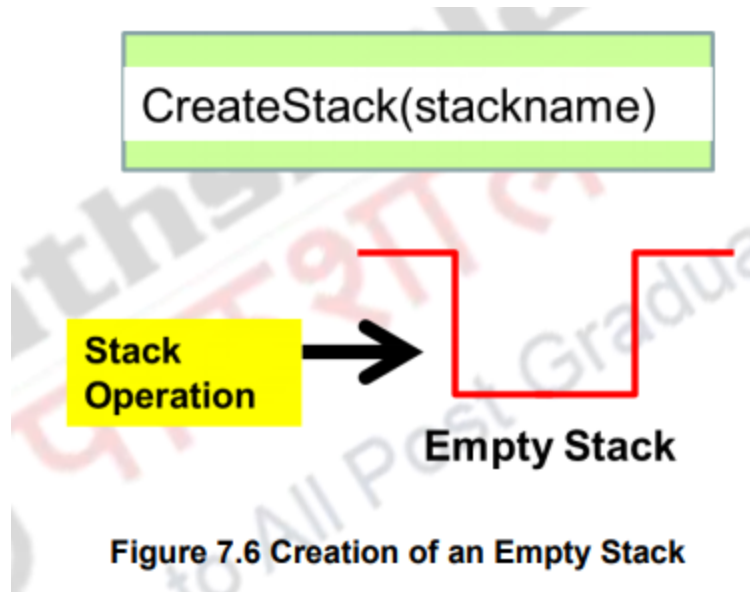


Figure 7.6 Creation of an Empty Stack

7.3.2.2 Checking whether Stack is Empty

Figure 7.7 shows the details of the operation that checks whether the Stack is empty. Here it is stated that the operation is Boolean, it has no preconditions and the post-condition is that a true value is returned if the stack is empty and false otherwise.

Determines whether a stack is empty

bool isEmpty() const;

Precondition: None

Postcondition:
Returns true if the stack is empty;
otherwise returns false

Figure 7.7 Details of the Operation – Checking whether Stack is Empty

7.3.2.3 Push Operation

The **push** operation inserts an item at the top of the stack (Figure 7.8). Here the push operation is called with the push(**stackName**,**dataitem**) into which addition is to be done and the element (**dataItem**) which is to be inserted. The figure shows the example of a stack before the push operation where the top points to the top item **79**. The item to be pushed is **40** and after the operation **40** is now at the top of the stack with top now pointing to **40**.

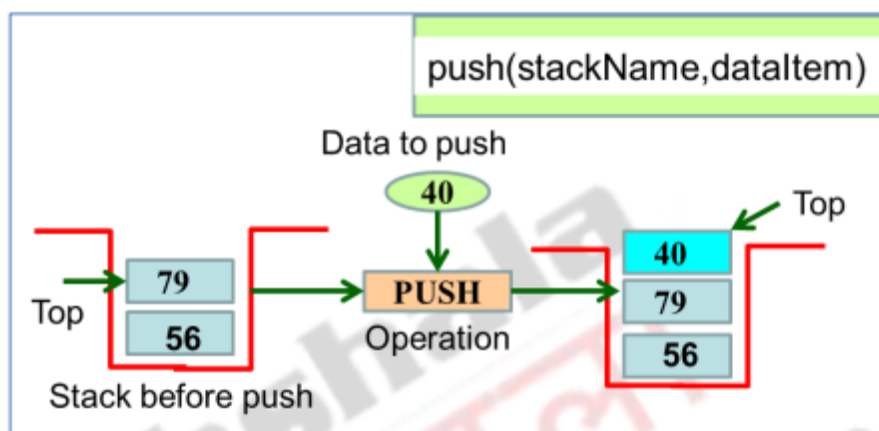


Figure 7.8 Push Operation

Figure 7.9 shows the definition of the push operation defined in another way. Here push is defined as a Boolean operation with input being the **newItem** of type **StackItem Type**. The precondition is that the **newItem** is the item to be added and post-conditions are that the operation will signal a true value if push is successful

and the **newItem** will then be on top of the stack. If the operation is unsuccessful (due to errors or exceptions such as the stack being full in the case of an array implementation) then the operation will signal a false value.

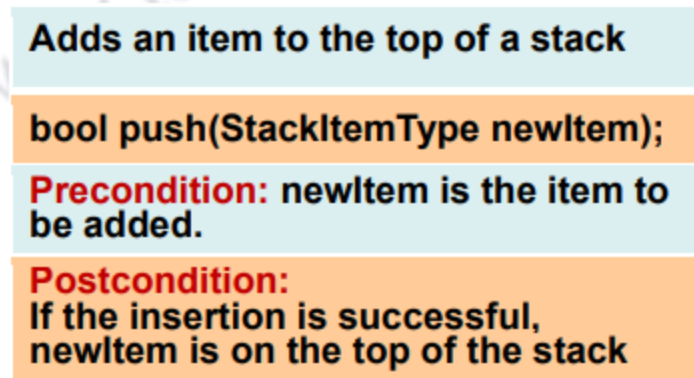


Figure 7.9 Details of the Push Operation

7.2.3.4 POP Operation

The **pop** operation removes an item from the top of the stack (Figure 7.10). Here the pop operation is called with the stack (**stackName**) from which deletion is to be done. The element **dataItem** is returned and is the item that has been removed. The figure shows the example of a stack before the pop operation with **40** on top of the stack which is then removed by the pop operation as dataItem. The Top now points to the element **79**. Note that pop can be defined as an operation that is not Boolean but an operation that returns the removed dataitem. In this case before dequeue operation can commence there is a need to check whether the stack is empty and signal an error if it is so.

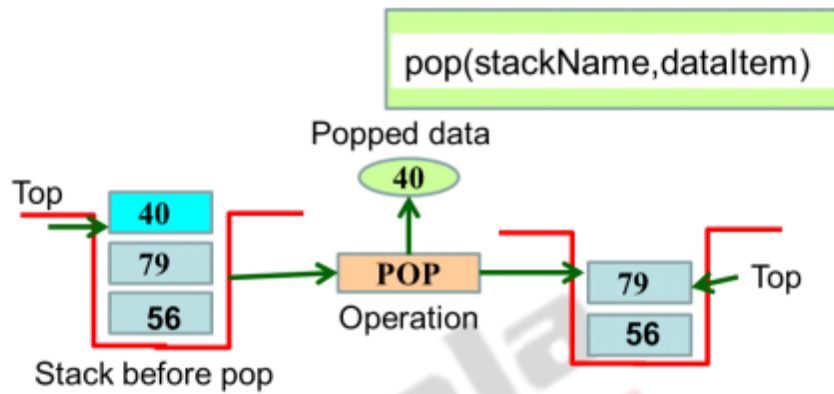


Figure 7.10 Pop Operation

Figure 7.11 shows the definition of the pop operation defined in another way. Here pop is defined as a Boolean operation with input being the access to the top of the stack (**stackTop**). There is no precondition and the post-conditions are that the operation will signal a true value if stack is not empty and the item at **stackTop** is removed. However if the stack is empty the operation signals a false value and **stackTop** remains unchanged.

Retrieves and removes the top of a stack
<code>bool pop(StackItemType& stackTop);</code>
Precondition: None.
Postcondition: • If the stack is not empty, stackTop contains the item and the item is removed and stackTop is changed. • However, if the stack is empty, deletion is impossible and stackTop is unchanged

Figure 7.11 Details of the Pop Operation

7.2.3.5 Top Operation

Retrieves the top element of the stack

```
bool top(StackItemType& stackTop) const;;
```

Precondition: None.

Postcondition:

- If the stack is not empty, **stackTop** contains the item that was added most recently.
- However, if the stack is empty, the operation fails and **stackTop** is unchanged.
- The stack is unchanged.

Figure 7.12 Details of the Top Operation

Figure 7.12 gives the details of the Boolean operation Top. In this case the input given is the same as Pop that is **stackTop** however this operation retrieves the element at the top without deleting it. There is no precondition and the post-conditions are that the operation will signal a true value if stack is not empty but **stackTop** that contains the element currently on top of the stack is returned. However if the stack is empty the operation signals a false value. In either case the stack remains unchanged.

7.2.4 Exceptions

Attempting the execution of an operation of ADT may sometimes cause error conditions or exceptions. Exceptions are said to be “thrown” by an operation that cannot be executed. In the case of Stack ADT, operations pop and top cannot be performed if the stack is empty. Attempting the execution of pop or top on an empty stack throws an EmptyStackException

7.3 Series of Stack Operations

In this section we have discussed a series of typical stack operations. However this figure must be understood only in terms of what happens in the stack since we have used a linked list type of visualization. This implementation of stacks will be discussed in future modules.

Figure 7.13 shows the stack at various stages during which a series of Stack ADT

operations are carried out. The first operation (Figure 7.13 (a)) to be carried out is the creation of an empty stack where count is set to zero and top is set to null. The second operation is a Push operation (Figure 7.13 (b)) where an element Green is inserted into the stack. Now count is set to one and Top points to the added element. The third operation is again a Push operation (Figure 7.13 (c)) where an element Blue is inserted into the stack. Now count is set to two and Top points to the newly added element (Blue). The fourth operation is a POP operation (Figure 7.13 (d)) where the element at the top of the stack (that is Blue) is removed. Now count is reset to one and Top once again points to Green. Finally we show the operation Destroy (Figure 7.13 (e)) where the entire stack is removed. These series of operations should give you an idea of how stacks work. Please note that we have not discussed the details of the implementation.

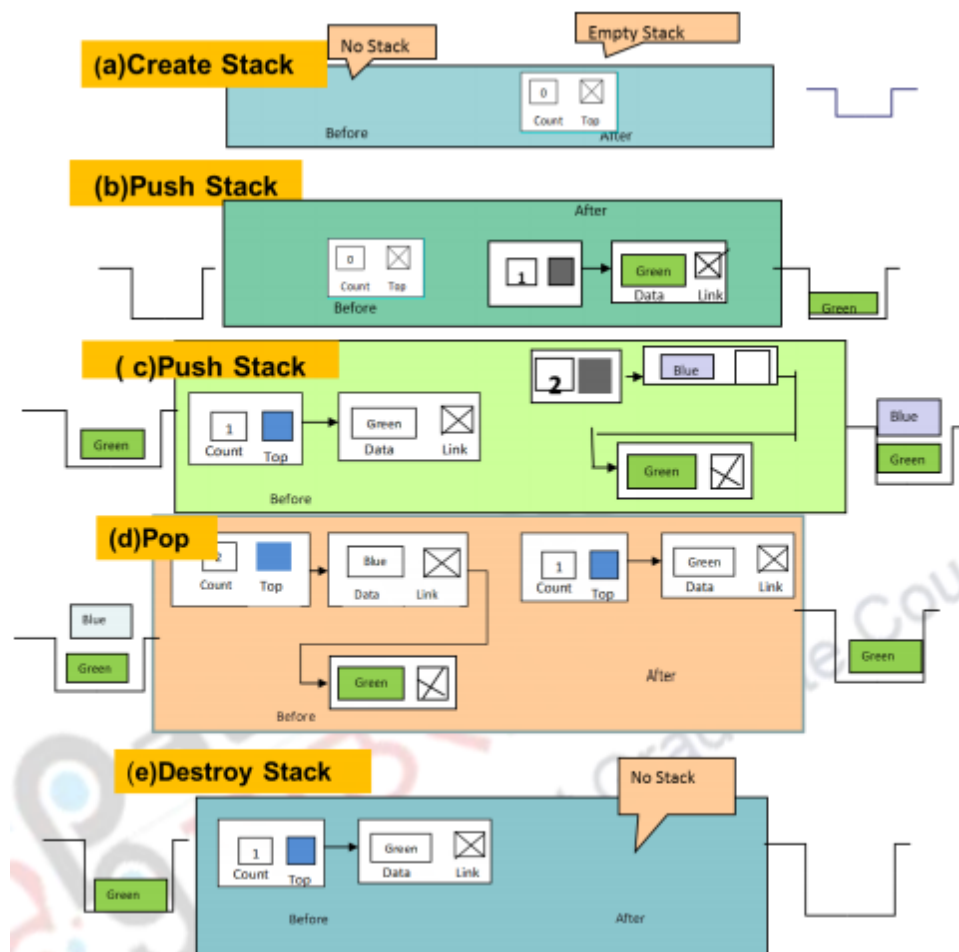


Figure 7.13 Series of Stack Operations

7.4 Applications of Stacks

Stack is one ADT that is widely used in many areas of computing. There are some direct applications of stacks. These include keeping track of Page-visited history in a Web browser, undoing the sequence of operations in a text editor, keeping track of the chain of method calls in the Java Virtual Machine or C++ runtime environment, etc.. There are also some indirect applications of stacks in the area of computing such as auxiliary data structure for algorithms and as a component of other data structures.

7.4.1 Uses of Stacks in Computing

Let us look at some of the uses of stacks in computing in more detail. Stacks are useful for any kind of problem involving Last-in-First-Out or **LIFO** data. One such example is in **Backtracking** which is a common situation in puzzles and games. In this context, we explore a path and find that we have reached a dead end, then we backtrack to find an alternative solution. When we do backtracking, we normally retract to the last position in which we have made a decision where there were other alternatives. Another typical application is in the development of **Browsers** where stacks are used to keep track of pages visited in a browser tab. Another application of stacks is in **Word Processors, editors** where stacks are used to check expressions or strings of text for matching parentheses / brackets e.g. if $(a == b) \{ c = (d + e) * f; \}$ and to implement *undo* operations in order to keep track of the most recent operations. Stacks are also used with **Markup languages (e.g. HTML, XML)**: which have formatting information (**tags**) that need matching (Figure 7.14)

e.g. `<HEAD>`
`<TITLE>Computer Science 1027a</TITLE> </HEAD>`

Figure 7.14 Tags of a Markup Language

Stacks are also in **Stack Calculators** to convert an **infix** expression to **postfix**, to make evaluation easier (we will be discussing this more in detail in future modules). The same conversion of infix expressions to postfix is carried out to make translation in **Compilers** of a high-level language such as Java or C to a lower level language easier is done using a stack. Another common stack – the **Call stack (Runtime stack)** is used by runtime system when methods are invoked, for method call / return processing (Figure 7.15). Stacks holds the “*call frame*” containing local variables, parameters, etc. The reason for using stacks in this context will also be discussed later.

e.g. main calls method1
method1 calls method 2
method 2 returns ...
Figure 7.15 Method Call and Return

7.5 Implementations of the ADT Stack

Before we finish this module about Stack ADT let us list some of the ways in which it can be implemented. Stacks can be implemented using (Figure 7.16):

- An array
- A linked list
- The ADT list

Since Stack is a restricted list, any list implementation could be used to implement a stack. When we use arrays, the stack is static that is the size of stack has to be fixed initially. On the other hand when we use linked lists, the stack is dynamic and the stack will never become full. In addition we will also see how to implement stacks using the ADT list. In future modules we will explore implementations based on array, linked list and ADT list

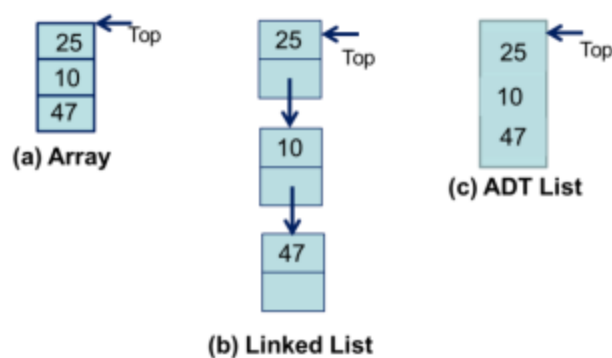


Figure 7.15 Implementations of Stacks

Summary

In this module we

- Discussed the concept of Stack
- Explained the Stack ADT
- Outlined most operations possible with the stack
- Discussed possible uses of the stack
- Listed possible implementations of the stack

you can view video on Stack ADT



Web Links
www2.latech.edu/~box/ds/chap4.ppt
www.csie.kuas.edu.tw/course/CS/old/english/ch-12.ppt
sce.uhcl.edu/shresthab/csci3033/Lecture6.ppt
www.cs.ccsu.edu/~neli/cs253/LECTURE3.PPT
http://younginc.site11.com/source/5895/fos0039.html
http://www.cse.unt.edu/~donr/courses/2050/Stacks.php
http://people.cs.pitt.edu/~chang/bookds/c05thu.html
Supporting & Reference Materials
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Adam Drozdek, "Data Structures and Algorithms in C++", Cengage; 4th edition, 2013
Alfred V. Aho and Jeffrey D. Ullman, "Data Structures and Algorithms", 1983
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014